



IMT Atlantique

Bretagne-Pays de la Loire
École Mines-Télécom



**CORS 2021
Annual Conference**

A Learning-based Iterated Local Search Algorithm for solving the Traveling Salesman Problem

Maryam KARIMI-MAMAGHAN
Patrick MEYER
Mehrdad MOHAMMADI
Bastien PASDELOUP

Combinatorial Optimization Problems (COPs)

- Discrete decision variables
- Finite search space
- Challenging to solve NP-hard COPs by exact algorithms

Meta-heuristics (MHs)

- Good substitutes for exact algorithms

Machine Learning (ML)

- Used **solely/ integrated** into MHs/exact algorithms for solving optimization problems
- ML techniques can be integrated into MHs for different purposes (Bengio et al. 2020, Talbi 2020, Karimi-Mamaghan et al. 2021)
- Enhance the performance of MHs by extracting knowledge

Operator Selection

- MHs use Local search and perturbation operators
- Non-stationary solution space
- Availability of many general and problem-specific operators
- Operators with different characteristics

Operator Selection

- MHs use Local search and perturbation operators
- Non-stationary solution space
- Availability of many general and problem-specific operators
- Operators with different characteristics

Operator selection strategies

- Pre-determined order
- Random selection
- Adaptive operator selection

This work

- Integrate a **ML technique** into a **MH algorithm** for solving **combinatorial optimization problems**
- Develop a new algorithm by integration of the **Q-learning algorithm** into the **iterated local search** meta-heuristic
- Purpose of integration is **adaptive operator selection**
- Present an application to the **traveling salesman problem**

1. Literature review
2. Background
 - 3.1. Iterated local search
 - 3.2. Q-learning
3. Proposed algorithm 1
 - 4.1. Algorithm design
 - 4.2. Experimental design
 - 4.3. Results
4. Proposed algorithm 2
5. Comparative results
6. Conclusion

Table 1. Relevant papers in the literature

Reference	Machine learning technique	Meta-heuristic	Operator	COP
Sakurai et al. (2010)	Q-learning	GA	Mutation & crossover	Traveling salesman problem
Handoko et al. (2014)	Q-learning	MA	Crossover	Quadratic assignment problem
dos Santos et al. (2014)	Q-learning	VNS	Local search	Traveling salesman problem
Buzdalova et al. (2014)	Q-learning	EA	Mutation & crossover	Traveling salesman problem
Gunawan et al. (2018)	Learning automata	ILS	Local search	Orienteering problem
Ahmadi et al. (2018)	Q-learning	GA	Mutation & crossover	Job sequencing problem
Mosadegh et al. (2020)	Q-learning	SA	Local search	Line sequencing problem

Contributions

- Integration of Q-learning into iterated local search algorithm
- Integration in 2 levels
 - Selection of local search
 - Selection of perturbation
- Proposing a new design of the Q-learning algorithm

Table 1. Relevant papers in the literature

Reference	Machine learning technique	Meta-heuristic	Operator	COP
Sakurai et al. (2010)	Q-learning	GA	Mutation & crossover	Traveling salesman problem
Handoko et al. (2014)	Q-learning	MA	Crossover	Quadratic assignment problem
dos Santos et al. (2014)	Q-learning	VNS	Local search	Traveling salesman problem
Buzdalova et al. (2014)	Q-learning	EA	Mutation & crossover	Traveling salesman problem
Gunawan et al. (2018)	Learning automata	ILS	Local search	Orienteering problem
Ahmadi et al. (2018)	Q-learning	GA	Mutation & crossover	Job sequencing problem
Mosadegh et al. (2020)	Q-learning	SA	Local search	Line sequencing problem

2. Background

2.1. Iterated Local Search (ILS)

7

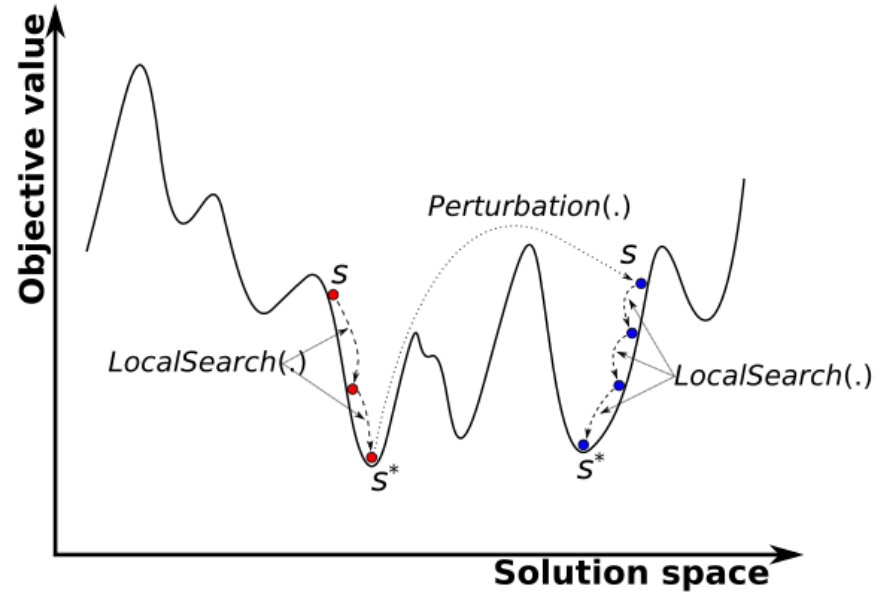
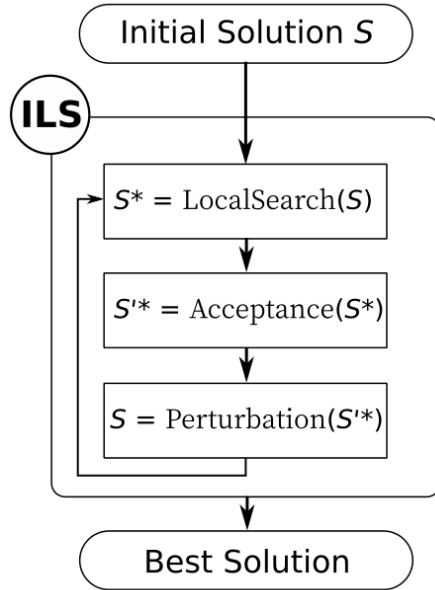


Figure 1. Iterated local search algorithm

2. Background

2.1. Iterated Local Search (ILS)

7

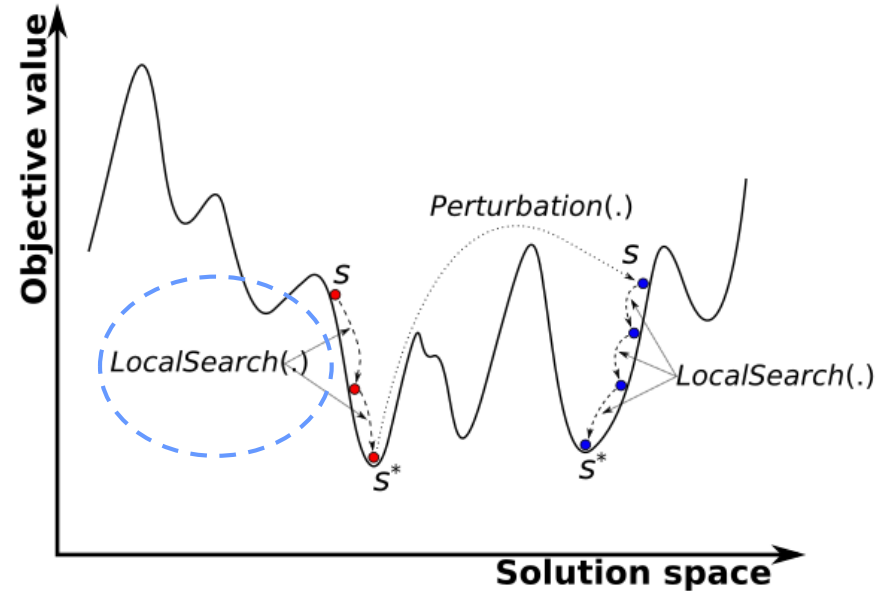
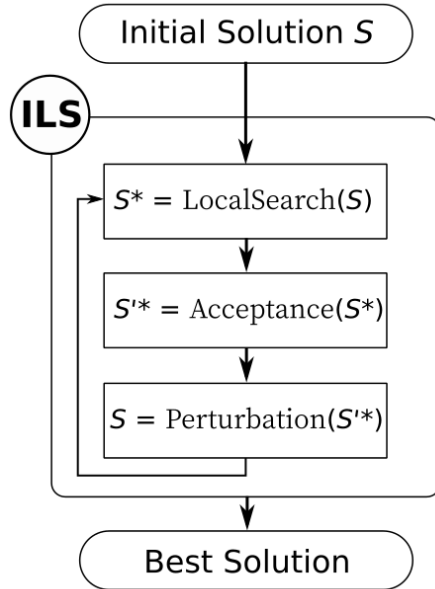


Figure 1. Iterated local search algorithm

2. Background

2.1. Iterated Local Search (ILS)

7

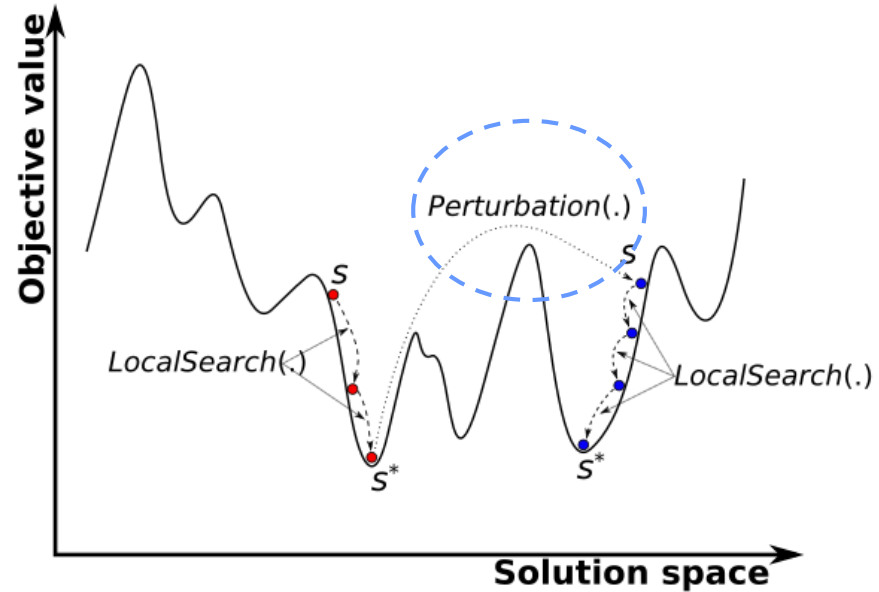
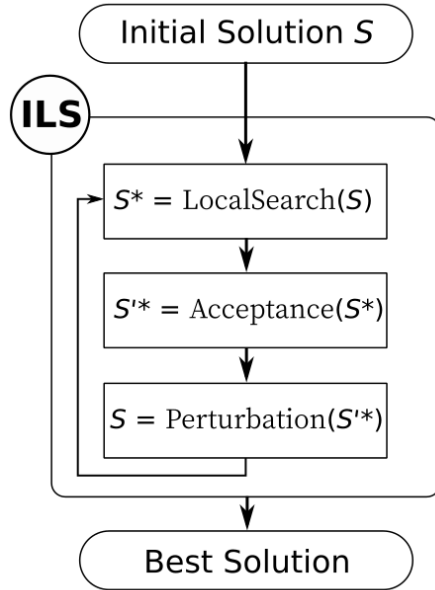


Figure 1. Iterated local search algorithm

The Q-Learning algorithm

- A reinforcement learning algorithm

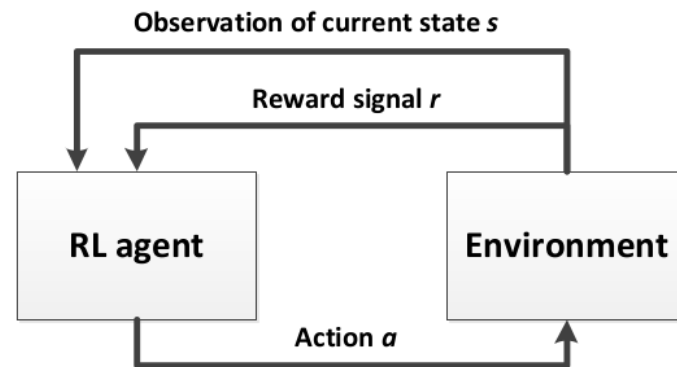


Figure 2. Reinforcement learning procedure

Wauters, et al., 2013. Boosting metaheuristic search using reinforcement learning. In Hybrid Metaheuristics (pp. 433-452). Springer, Berlin, Heidelberg.

The Q-Learning algorithm

- A reinforcement learning algorithm
- Model-free algorithm

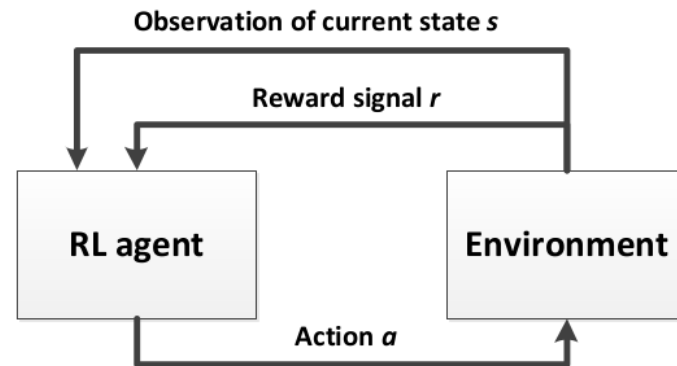


Figure 2. Reinforcement learning procedure

Wauters, et al., 2013. Boosting metaheuristic search using reinforcement learning. In Hybrid Metaheuristics (pp. 433-452). Springer, Berlin, Heidelberg.

The Q-Learning algorithm

- A reinforcement learning algorithm
- Model-free algorithm
- Define states, actions, and Q-value for each (state, action)

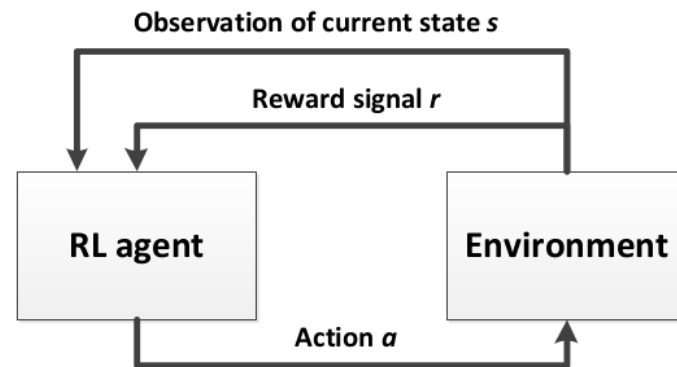


Figure 2. Reinforcement learning procedure

Wauters, et al., 2013. Boosting metaheuristic search using reinforcement learning. In Hybrid Metaheuristics (pp. 433-452). Springer, Berlin, Heidelberg.

The Q-Learning algorithm

- A reinforcement learning algorithm
- Model-free algorithm
- Define states, actions, and Q-value for each (state, action)

$$Q(s, a) = Q(s, a) + \alpha[r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

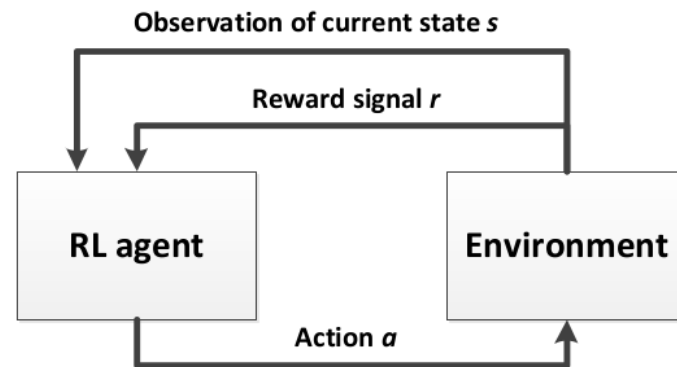


Figure 2. Reinforcement learning procedure

Wauters, et al., 2013. Boosting metaheuristic search using reinforcement learning. In Hybrid Metaheuristics (pp. 433-452). Springer, Berlin, Heidelberg.

The Q-Learning algorithm

- A reinforcement learning algorithm
- Model-free algorithm
- Define states, actions, and Q-value for each (state, action)

$$Q(s, a) = Q(s, a) + \alpha[r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

- s, s' : state
- a : the action performed in state s
- r : the reinforcement signal (return or reward) received after performing a in s
- $0 \leq \alpha < 1$: the learning rate
- $0 \leq \gamma < 1$: the discount factor

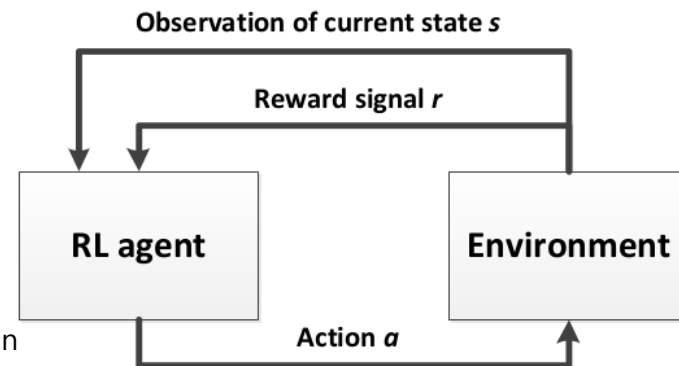


Figure 2. Reinforcement learning procedure

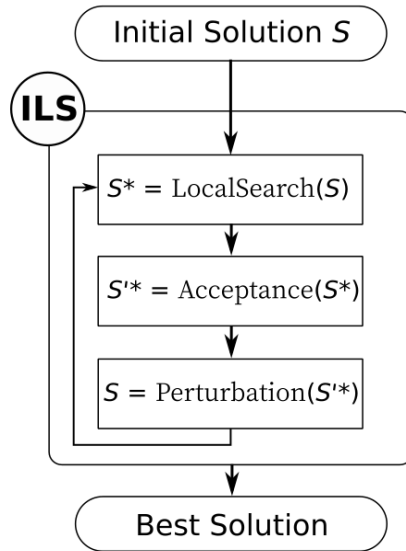
Wauters, et al., 2013. Boosting metaheuristic search using reinforcement learning. In Hybrid Metaheuristics (pp. 433-452). Springer, Berlin, Heidelberg.

3. Proposed algorithm 1

9

3.1. Algorithm design

Iterated local search



3. Proposed algorithm 1

10

3.1. Algorithm design

Iterated local search with Multiple local search

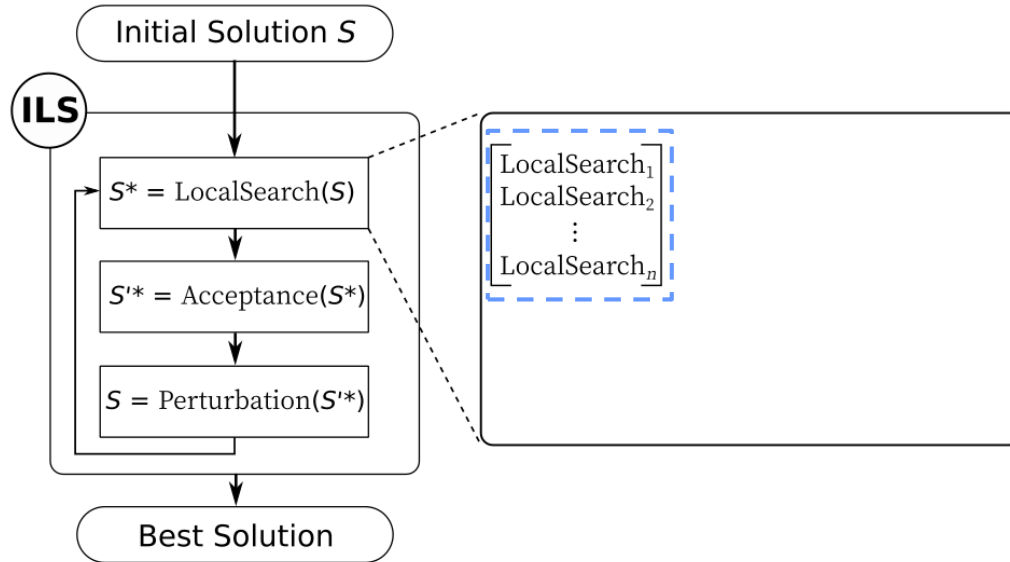


Figure 3. The framework of algorithm 1

3. Proposed algorithm 1

10

3.1. Algorithm design

Iterated local search with Multiple local search

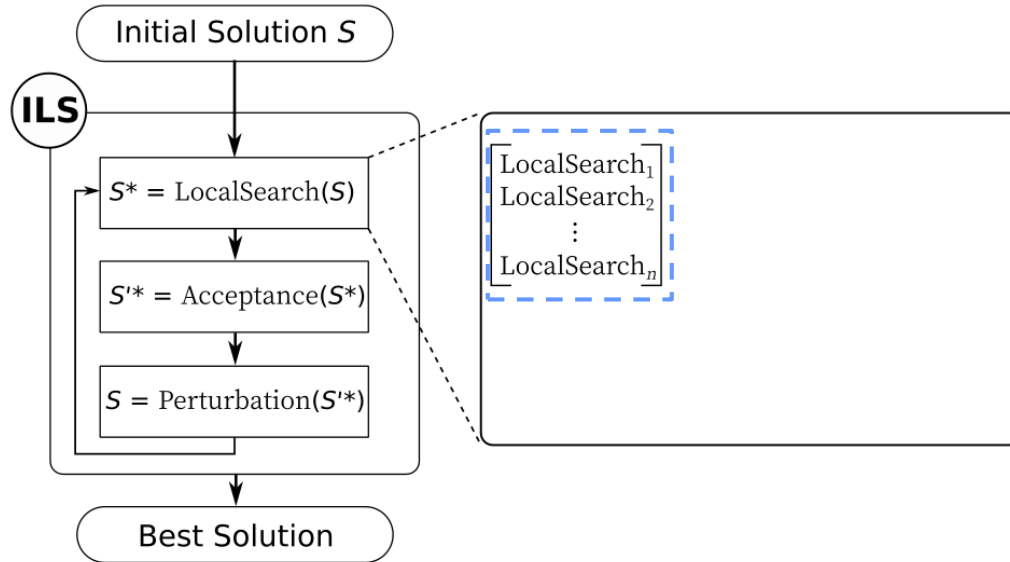


Figure 3. The framework of algorithm 1

ILS

- **Local Search operators**
 - Basic 2opt
 - Improved 2opt
 - Insertion
- **Perturbation operator**
 - Double-bridge

3. Proposed algorithm 1

13

3.1. Algorithm design

Integration of Q-learning into Iterated local search

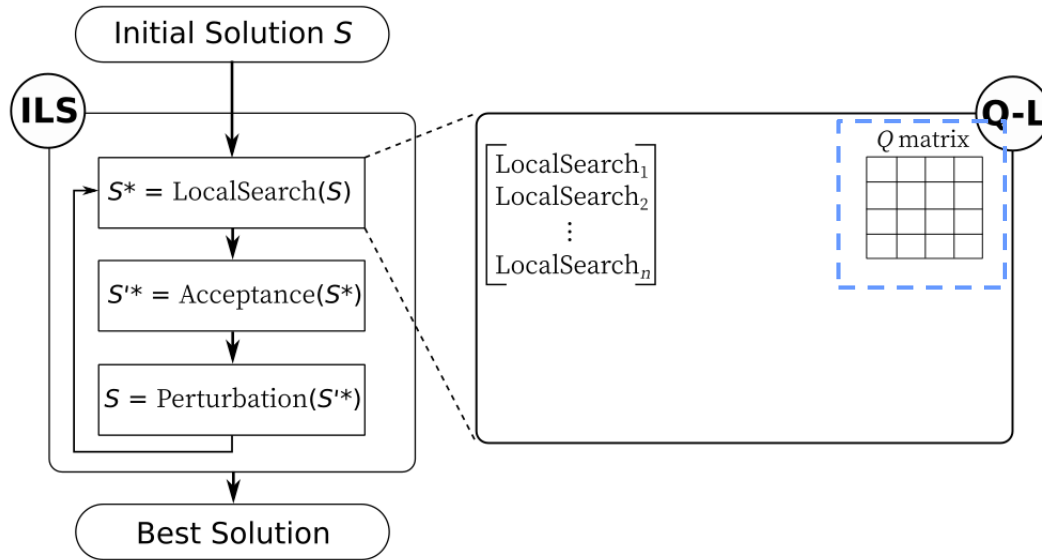


Figure 3. The framework of algorithm 1

3.1. Algorithm design

Integration of Q-learning into Iterated local search

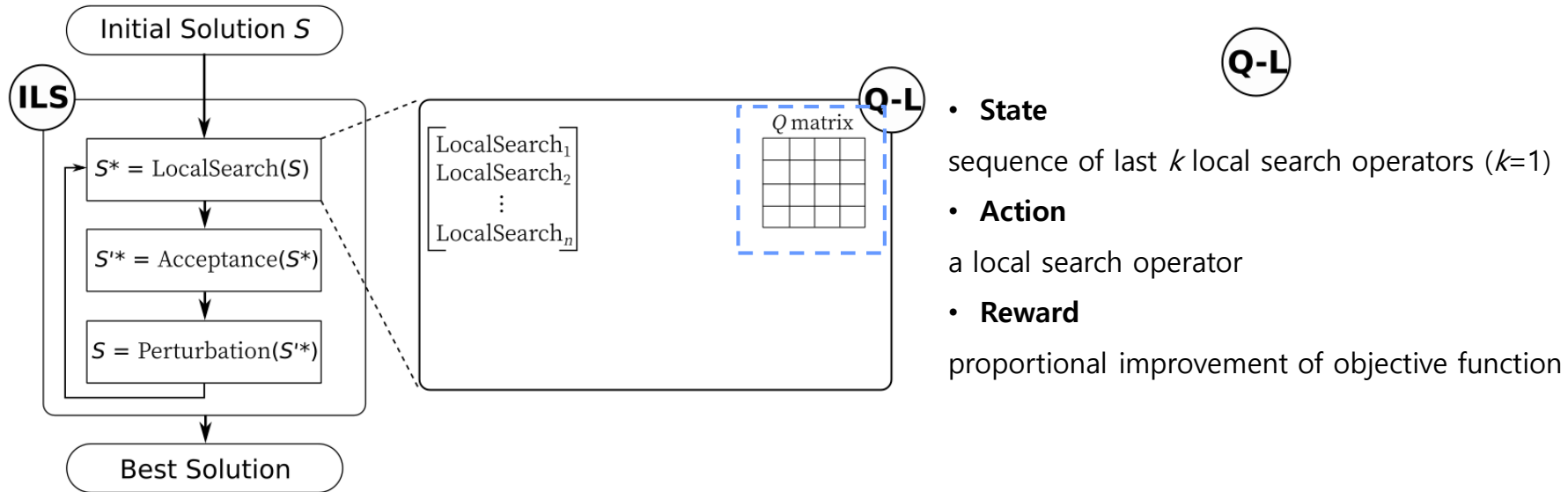


Figure 3. The framework of algorithm 1

3.1. Algorithm design

Integration of Q-learning into Iterated local search

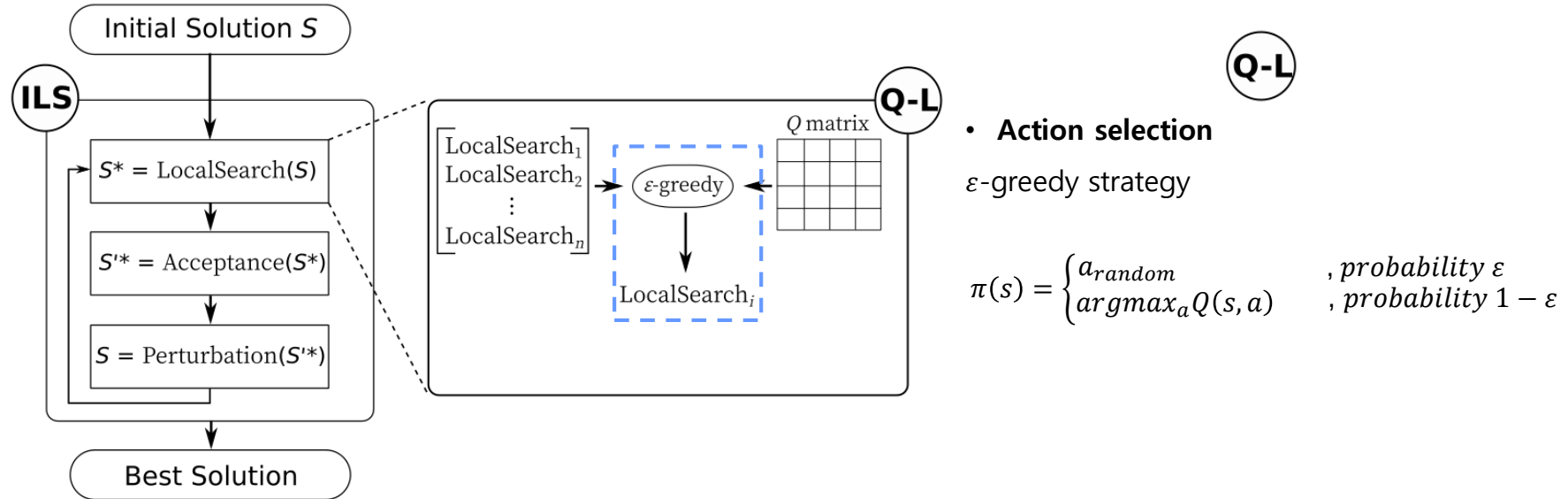


Figure 3. The framework of algorithm 1

3. Proposed algorithm 1

15

3.1. Algorithm design

Integration of Q-learning into Iterated local search

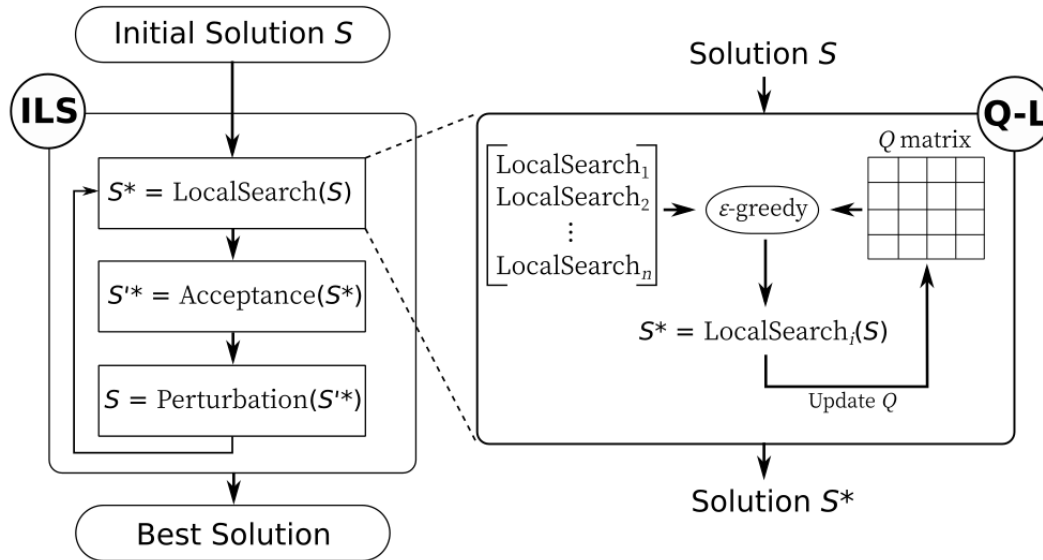


Figure 3. The framework of algorithm 1

3. Proposed algorithm 1

15

3.1. Algorithm design

Integration of Q-learning into Iterated local search

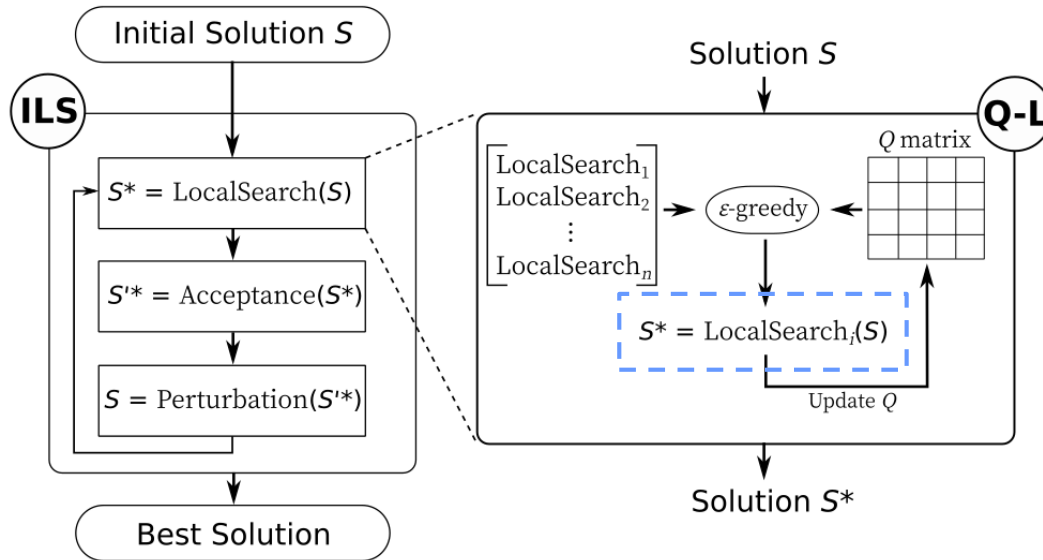


Figure 3. The framework of algorithm 1

3. Proposed algorithm 1

15

3.1. Algorithm design

Integration of Q-learning into Iterated local search

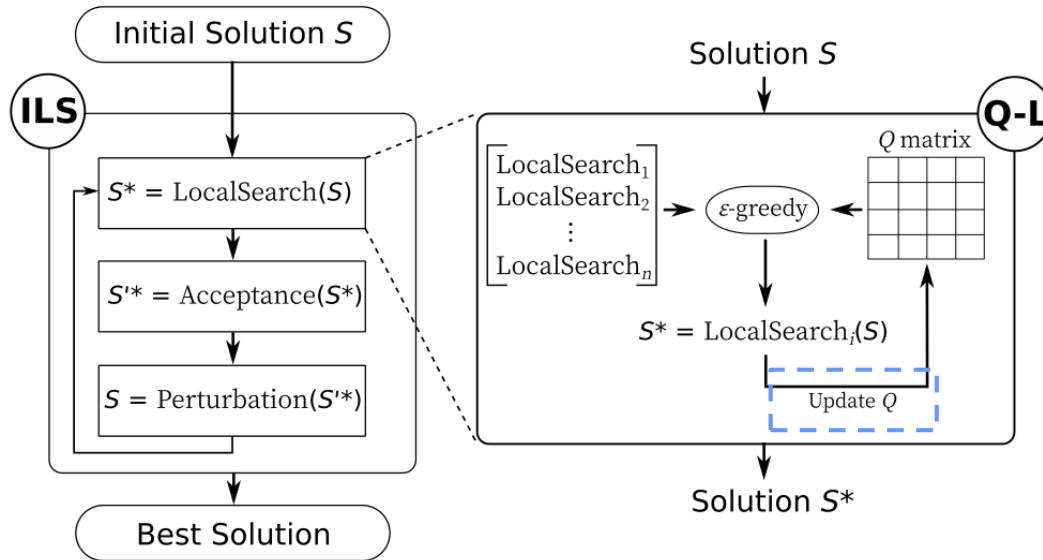


Figure 3. The framework of algorithm 1

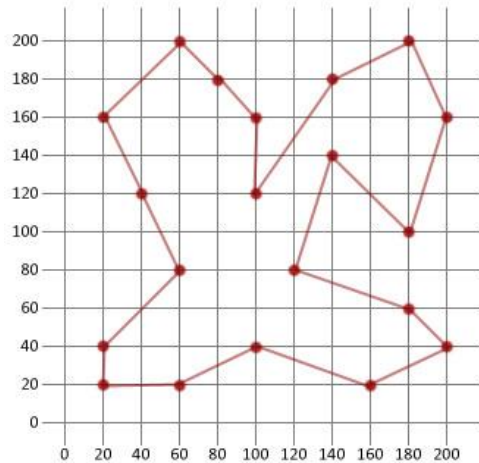
3. Proposed algorithm 1

16

3.2. Experimental design

Traveling Salesman Problem (TSP)

- Data from TSPLIB
- Instances with 50-2000 cities



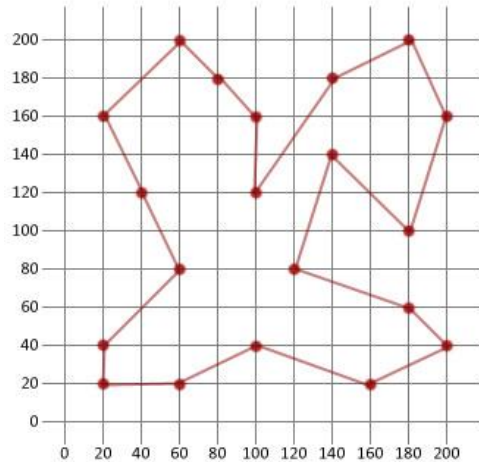
3. Proposed algorithm 1

16

3.2. Experimental design

Traveling Salesman Problem (TSP)

- Data from TSPLIB
- Instances with 50-2000 cities



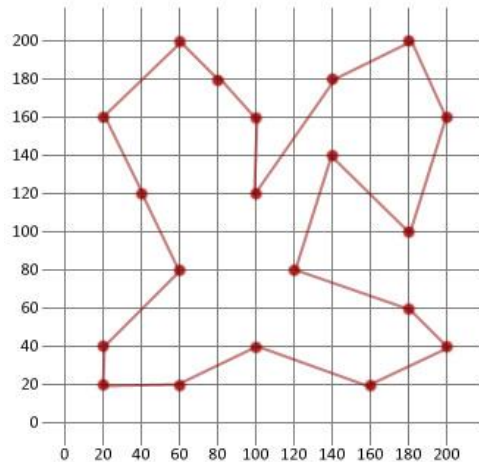
Parameter tuning (Design of Experiments)

- *Parameters:*
 - ϵ (*Epsilon*)
 - β (*Epsilon decay*)
 - α (*Learning rate*)
 - γ (*Discount factor*)

3.2. Experimental design

Traveling Salesman Problem (TSP)

- Data from TSPLIB
- Instances with 50-2000 cities



Parameter tuning (Design of Experiments)

- *Parameters:*
 - ϵ (*Epsilon*)
 - β (*Epsilon decay*)
 - α (*Learning rate*)
 - γ (*Discount factor*)

Performance Criteria

- Quality of solutions (RPD)

$$RPD = \frac{OF - OF^*}{OF^*} \times 100$$

- Computational effort (time)
- Convergence behavior

Performance criteria

- Quality of solutions (RPD%)
- Computational effort (time)

3.3. Results

Performance criteria

- Quality of solutions (RPD%)
- Computational effort (time)

Table 2. Results of algorithm 1

Instance	Optimal	Best RPD(%)	Best time (s)	Average RPD(%)	Average time (s)
berlin52	7542	0	0.1	0	0.7
st70	675	0	2.4	0.037	9.9
kroA100	21282	0	0.3	0.005	10.2
rd100	7910	0	9.3	0.173	25.9
lin105	14379	0	0.5	0	15.2
pr124	59030	0	4.7	0.004	34.1
ch130	6110	0.262	42.9	0.546	73.4
ch150	6528	0.077	50.5	0.465	50.8
u159	42080	0	5.8	0	55.4
d198	15780	0.165	286.8	0.263	220.9
kroA200	29368	0.051	97.3	0.352	237.5
ts225	126643	0	1.1	0	48.0
pr264	49135	0	303.5	0.402	207.1
a280	2579	0	451.8	0.587	396.8
pr299	48191	0.151	734.7	0.805	704.3
lin318	42029	0.895	737.2	1.559	688.7
fl417	11861	0.430	605.5	0.696	796.1
pr439	107217	0.755	840.1	2.308	778.2
pcb442	50778	1.061	747.7	1.568	758.4
d493	35002	1.451	16.5	2.135	643.4
vm1084	239297	3.025	1461.4	4.217	2012.2
d1291	50801	3.173	782.4	4.295	1835.3
u1817	57201	4.142	254.0	4.858	664.4
u2152	64253	3.836	235.3	4.841	919.8

3.3. Results

Performance criteria

- Quality of solutions (RPD%)
- Computational effort (time)

Table 2. Results of algorithm 1

Instance	Optimal	Best RPD(%)	Best time (s)	Average RPD(%)	Average time (s)
berlin52	7542	0	0.1	0	0.7
st70	675	0	2.4	0.037	9.9
kroA100	21282	0	0.3	0.005	10.2
rd100	7910	0	9.3	0.173	25.9
lin105	14379	0	0.5	0	15.2
pr124	59030	0	4.7	0.004	34.1
ch130	6110	0.262	42.9	0.546	73.4
ch150	6528	0.077	50.5	0.465	50.8
u159	42080	0	5.8	0	55.4
d198	15780	0.165	286.8	0.263	220.9
kroA200	29368	0.051	97.3	0.352	237.5
ts225	126643	0	1.1	0	48.0
pr264	49135	0	303.5	0.402	207.1
a280	2579	0	451.8	0.587	396.8
pr299	48191	0.151	734.7	0.805	704.3
lin318	42029	0.895	737.2	1.559	688.7
fl417	11861	0.430	605.5	0.696	796.1
pr439	107217	0.755	840.1	2.308	778.2
pcb442	50778	1.061	747.7	1.568	758.4
d493	35002	1.451	16.5	2.135	643.4
vm1084	239297	3.025	1461.4	4.217	2012.2
d1291	50801	3.173	782.4	4.295	1835.3
u1817	57201	4.142	254.0	4.858	664.4
u2152	64253	3.836	235.3	4.841	919.8

3.3. Results

Performance criteria

- Quality of solutions (RPD%)
- Computational effort (time)

Table 2. Results of algorithm 1

Instance	Optimal	Best RPD(%)	Best time (s)	Average RPD(%)	Average time (s)
berlin52	7542	0	0.1	0	0.7
st70	675	0	2.4	0.037	9.9
kroA100	21282	0	0.3	0.005	10.2
rd100	7910	0	9.3	0.173	25.9
lin105	14379	0	0.5	0	15.2
pr124	59030	0	4.7	0.004	34.1
ch130	6110	0.262	42.9	0.546	73.4
ch150	6528	0.077	50.5	0.465	50.8
u159	42080	0	5.8	0	55.4
d198	15780	0.165	286.8	0.263	220.9
kroA200	29368	0.051	97.3	0.352	237.5
ts225	126643	0	1.1	0	48.0
pr264	49135	0	303.5	0.402	207.1
a280	2579	0	451.8	0.587	396.8
pr299	48191	0.151	734.7	0.805	704.3
lin318	42029	0.895	737.2	1.559	688.7
fl417	11861	0.430	605.5	0.696	796.1
pr439	107217	0.755	840.1	2.308	778.2
pcb442	50778	1.061	747.7	1.568	758.4
d493	35002	1.451	16.5	2.135	643.4
vm1084	239297	3.025	1461.4	4.217	2012.2
d1291	50801	3.173	782.4	4.295	1835.3
u1817	57201	4.142	254.0	4.858	664.4
u2152	64253	3.836	235.3	4.841	919.8

4. Proposed algorithm 2

20

4.1. Algorithm design

Integration of Q-learning into Iterated local search

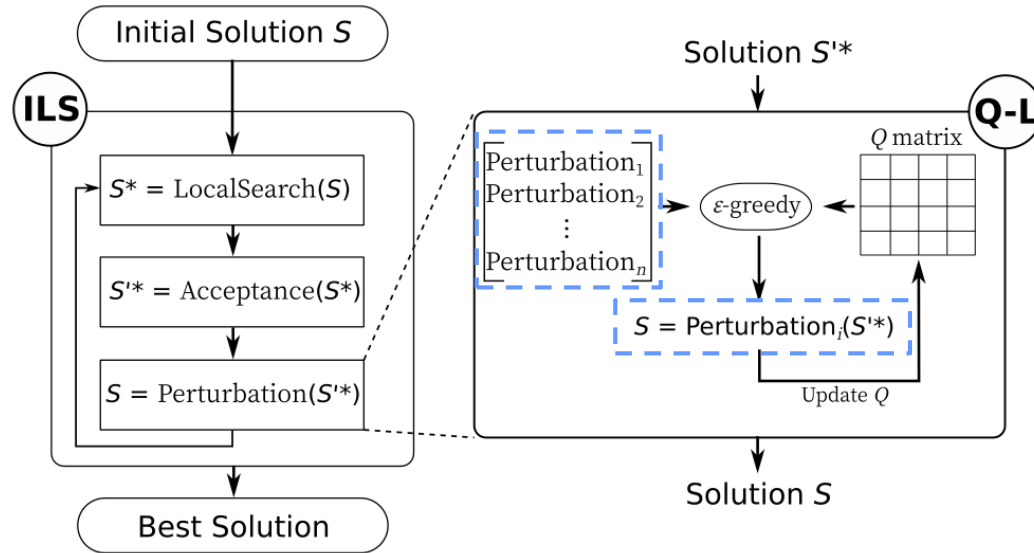


Figure 4. The framework of algorithm 2

Integration of Q-learning into Iterated local search

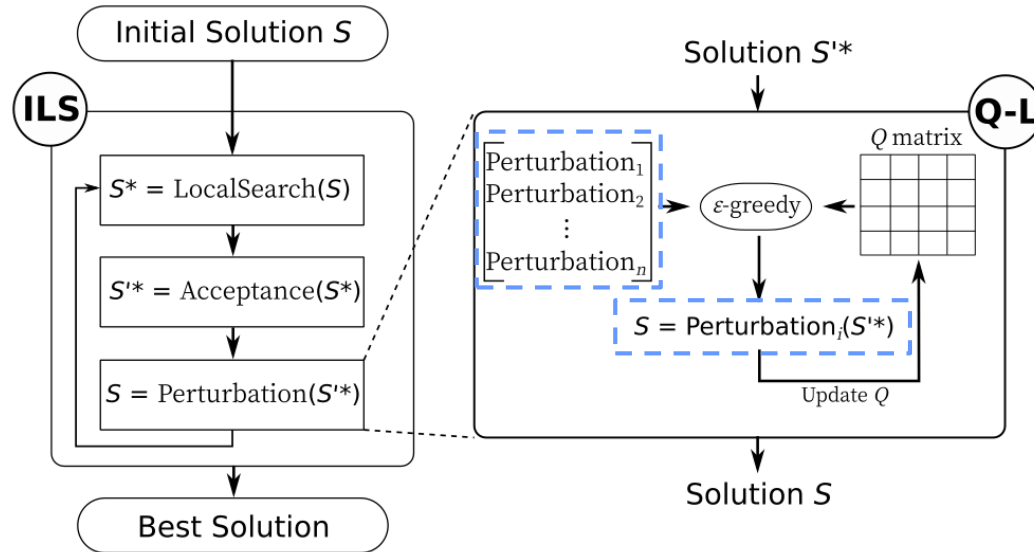


Figure 4. The framework of algorithm 2

ILS

- **Perturbation operators**
 - Double-bridge
 - Shuffle-sequence
 - Reversion-sequence
- **Local search operator**
 - Improved 2opt

4.1. Algorithm design

Integration of Q-learning into Iterated local search

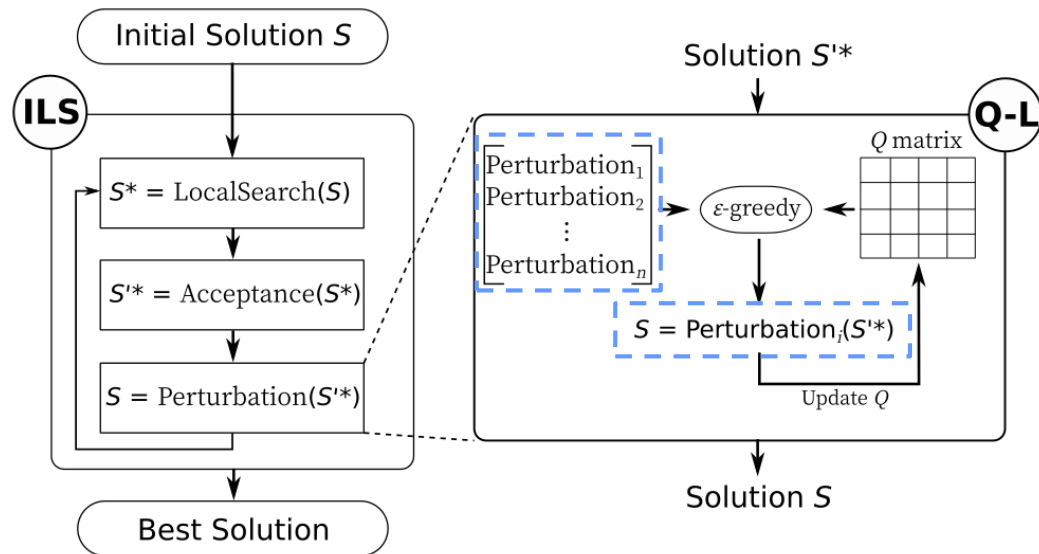


Figure 4. The framework of algorithm 2

ILS

- **Perturbation operators**
 - Double-bridge
 - Shuffle-sequence
 - Reversion-sequence
- **Local search operator**
 - Improved 2opt

Q-L

- **State**
1: if an improvement occurs in the best found solution
0: otherwise
- **Action**
(perturbation type, repetition number)
- **Reward**
proportional improvement of objective function over an **episode**

4.2. Results

Performance criteria

- Quality of solutions (RPD%)
- Computational effort (time)

Table 3. Results of algorithm 2

Instance	Optimal	Best RPD(%)	Best time (s)	Average RPD(%)	Average time (s)
berlin52	7542	0	0	0	0.2
st70	675	0	0.7	0.109	4.5
kroA100	21282	0	1.3	0	7.3
rd100	7910	0	3.6	0.201	14.2
lin105	14379	0	0.4	0	4.5
pr124	59030	0	1.8	0	8.7
ch130	6110	0	17.4	0.359	40.4
ch150	6528	0	19.8	0.374	33.2
u159	42080	0	9.6	0.112	42.9
d198	15780	0.057	149.4	0.180	147.5
kroA200	29368	0	117.3	0.150	194.8
ts225	126643	0	56.5	0.002	108.9
pr264	49135	0	61.4	0.172	135.8
a280	2579	0	246.4	0.498	276.5
pr299	48191	0	453.1	0.274	440.7
lin318	42029	0.302	502.4	0.795	479.6
fl417	11861	0.211	574.5	0.339	553.0
pr439	107217	0.438	573.9	1.392	528.2
pcb442	50778	0.640	518.4	1.294	521.6
d493	35002	0.923	571.5	1.499	587.8
vm1084	239297	3.061	2729.6	3.717	2608.3
d1291	50801	2.281	1985.9	3.418	2060.1
u1817	57201	3.449	412.1	4.366	1416
u2152	64253	3.947	1967.5	4.531	1022.3

4.2. Results

Performance criteria

- Quality of solutions (RPD%)
- Computational effort (time)

Table 3. Results of algorithm 2

Instance	Optimal	Best RPD(%)	Best time (s)	Average RPD(%)	Average time (s)
berlin52	7542	0	0	0	0.2
st70	675	0	0.7	0.109	4.5
kroA100	21282	0	1.3	0	7.3
rd100	7910	0	3.6	0.201	14.2
lin105	14379	0	0.4	0	4.5
pr124	59030	0	1.8	0	8.7
ch130	6110	0	17.4	0.359	40.4
ch150	6528	0	19.8	0.374	33.2
u159	42080	0	9.6	0.112	42.9
d198	15780	0.057	149.4	0.180	147.5
kroA200	29368	0	117.3	0.150	194.8
ts225	126643	0	56.5	0.002	108.9
pr264	49135	0	61.4	0.172	135.8
a280	2579	0	246.4	0.498	276.5
pr299	48191	0	453.1	0.274	440.7
lin318	42029	0.302	502.4	0.795	479.6
fl417	11861	0.211	574.5	0.339	553.0
pr439	107217	0.438	573.9	1.392	528.2
pcb442	50778	0.640	518.4	1.294	521.6
d493	35002	0.923	571.5	1.499	587.8
vm1084	239297	3.061	2729.6	3.717	2608.3
d1291	50801	2.281	1985.9	3.418	2060.1
u1817	57201	3.449	412.1	4.366	1416
u2152	64253	3.947	1967.5	4.531	1022.3

4.2. Results

Performance criteria

- Quality of solutions (RPD%)
- Computational effort (time)

Table 3. Results of algorithm 2

Instance	Optimal	Best RPD(%)	Best time (s)	Average RPD(%)	Average time (s)
berlin52	7542	0	0	0	0.2
st70	675	0	0.7	0.109	4.5
kroA100	21282	0	1.3	0	7.3
rd100	7910	0	3.6	0.201	14.2
lin105	14379	0	0.4	0	4.5
pr124	59030	0	1.8	0	8.7
ch130	6110	0	17.4	0.359	40.4
ch150	6528	0	19.8	0.374	33.2
u159	42080	0	9.6	0.112	42.9
d198	15780	0.057	149.4	0.180	147.5
kroA200	29368	0	117.3	0.150	194.8
ts225	126643	0	56.5	0.002	108.9
pr264	49135	0	61.4	0.172	135.8
a280	2579	0	246.4	0.498	276.5
pr299	48191	0	453.1	0.274	440.7
lin318	42029	0.302	502.4	0.795	479.6
fl417	11861	0.211	574.5	0.339	553.0
pr439	107217	0.438	573.9	1.392	528.2
pcb442	50778	0.640	518.4	1.294	521.6
d493	35002	0.923	571.5	1.499	587.8
vm1084	239297	3.061	2729.6	3.717	2608.3
d1291	50801	2.281	1985.9	3.418	2060.1
u1817	57201	3.449	412.1	4.366	1416
u2152	64253	3.947	1967.5	4.531	1022.3

Benchmark algorithms:

- Single ILS (S-ILS)
- Random ILS (R-ILS)

Table 4. Comparative results of the proposed algorithms

	Algorithm 1				Algorithm 2			
	R-ILS	S-ILS			R-ILS	S-ILS		
		1	2	3		1	2	3
Average	1.19	1.16	1.41	3.12	1.58	2.71	1.49	1.61

Benchmark algorithms:

- Single ILS (S-ILS)
- Random ILS (R-ILS)

Table 4. Comparative results of the proposed algorithms

	Algorithm 1				Algorithm 2			
	R-ILS	S-ILS			R-ILS	S-ILS		
		1	2	3		1	2	3
Average	1.19	1.16	1.41	3.12	1.58	2.71	1.49	1.61

Worst-case complexity

- Algorithm 1: $O(I(4N^3 + A))$
- R-ILS: $O(I(4N^3))$
- Algorithm 2: $O(I(N^3 + A))$
- R-ILS: $O(I(N^3))$

Benchmark algorithms:

- Single ILS (S-ILS)
- Random ILS (R-ILS)

Table 4. Comparative results of the proposed algorithms

	Algorithm 1				Algorithm 2			
	R-ILS	S-ILS			R-ILS	S-ILS		
		1	2	3		1	2	3
Average	1.19	1.16	1.41	3.12	1.58	2.71	1.49	1.61

Worst-case complexity

- Algorithm 1: $O(I(4N^3 + A))$

- R-ILS: $O(I(4N^3))$

- Overhead of Q-learning: $O(I \times A)$

- Algorithm 2: $O(I(N^3 + A))$

- R-ILS: $O(I(N^3))$

- Overhead of Q-learning: $O(I \times A)$

Conclusion

- Proposed algorithms vs S-ILS and R-ILS
 - ILS with **Multiple local search/perturbation operators** outperforms single operator
 - Integration of **Q-learning** for operator selection has an added value without imposing significant extra complexity overhead
 - Integration of the knowledge from the Q-learning algorithm into the operator selection mechanism of the ILS algorithm is very efficient

Conclusion

- Proposed algorithms vs S-ILS and R-ILS
 - ILS with **Multiple local search/perturbation operators** outperforms single operator
 - Integration of **Q-learning** for operator selection has an added value without imposing significant extra complexity overhead
 - Integration of the knowledge from the Q-learning algorithm into the operator selection mechanism of the ILS algorithm is very efficient
- Algorithm 1 vs Algorithm 2
 - Integration of Q-learning significantly enhances the exploration ability of Algorithm 2
 - Q-learning is very useful when the **operators are competitive**

Perspective

- Solving COPs with several competitive operators

Thanks for your
attention

- Ahmadi, E., Goldengorin, B., Süer, G. A., & Mosadegh, H. (2018). A hybrid method of 2-TSP and novel learning-based GA for job sequencing and tool switching problem. *Applied Soft Computing*, 65, 214-229.
- Buzdalova, A., Kononov, V., & Buzdalov, M. (2014, July). Selecting evolutionary operators using reinforcement learning: Initial explorations. In *Proceedings of the Companion Publication of the 2014 Annual Conference on Genetic and Evolutionary Computation* (pp. 1033-1036).
- Dos Santos, J. P. Q., de Melo, J. D., Neto, A. D. D., & Aloise, D. (2014). Reactive search strategies using reinforcement learning, local search algorithms and variable neighborhood search. *Expert Systems with Applications*, 41(10), 4939-4949.
- Gunawan, A., Lau, H. C., & Lu, K. (2018). ADOPT: combining parameter tuning and adaptive operator ordering for solving a class of orienteering problems. *Computers & Industrial Engineering*, 121, 82-96.
- Handoko, S. D., Nguyen, D. T., Yuan, Z., & Lau, H. C. (2014, July). Reinforcement learning for adaptive operator selection in memetic search applied to quadratic assignment problem. In *Proceedings of the Companion Publication of the 2014 Annual Conference on Genetic and Evolutionary Computation* (pp. 193-194).
- Karimi-Mamaghan, M., Mohammadi, M., Meyer, P., Karimi-Mamaghan, A. M., & Talbi, E. G. (2021). Machine Learning at the service of Meta-heuristics for solving Combinatorial Optimization Problems: A state-of-the-art. *European Journal of Operational Research*.
- Mosadegh, H., Ghomi, S. F., & Süer, G. A. (2020). Stochastic mixed-model assembly line sequencing problem: Mathematical modeling and Q-learning based simulated annealing hyper-heuristics. *European Journal of Operational Research*, 282(2), 530-544.
- Sakurai, Y., Takada, K., Kawabe, T., & Tsuruta, S. (2010, December). A method to control parameters of evolutionary algorithms by using reinforcement learning. In *2010 Sixth International Conference on Signal-Image Technology and Internet Based Systems* (pp. 74-79). IEEE:
- Song, H., Triguero, I., & Özcan, E. (2019). A review on the self and dual interactions between machine learning and optimisation. *Progress in Artificial Intelligence*, 8(2), 143-165.
- Talbi, E. G. (2009). *Metaheuristics: from design to implementation* (Vol. 74). John Wiley & Sons.
- TSPLIB: <http://elib.zib.de/pub/mp-testdata/tsp/tsplib/tsplib.html>
- Wauters, T., Verbeeck, K., De Causmaecker, P., & Berghe, G. V. (2013). Boosting metaheuristic search using reinforcement learning. In *Hybrid Metaheuristics* (pp. 433-452). Springer, Berlin, Heidelberg

3. Proposed algorithm 1

11

3.1. Algorithm Design (Operators)

Basic 2opt

Step 1. Identify the first improving moves

Move 1, Improvement: 80

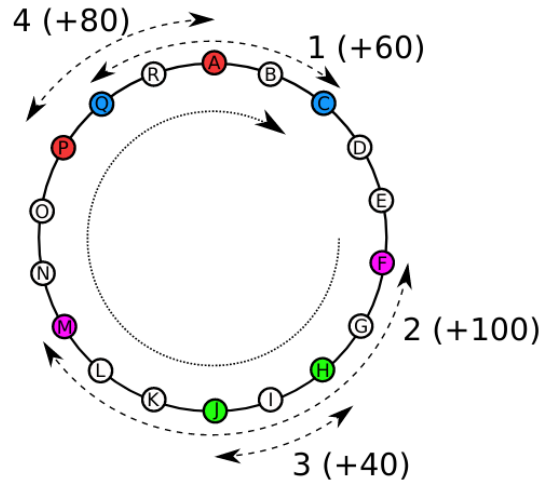


Figure 5. A complete tour of cities

Developed 2opt

Step 1. Identify the improving moves

Move 1, Improvement: 80

Move 2, Improvement: 200

Move 3, Improvement: 60

Move 4, Improvement: 100

Step 2. Sort the improving moves

Move 2

Move 4

Move 1

Move 3

Step 3. Perform independent moves simultaneously

Move 2 ✓

Move 4 ✓

Move 1 ✗

Move 3 ✓

3. Proposed algorithm 1

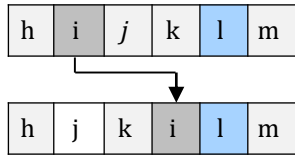
12

3.1. Algorithm Design (Operators)

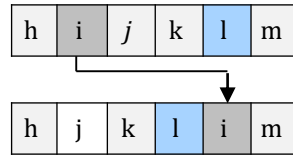
Insertion

4 insertion moves:

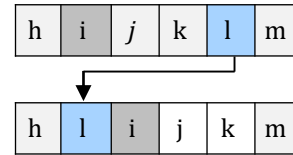
- Forward-left



- Forward-right



- Backward-left



- Backward-right

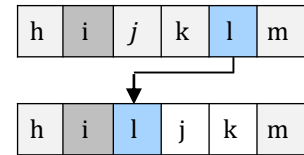


Figure 6. 4-move Insertion operator procedure